

Tarea XII - BFS y DFS

Programación III

Integrantes:

ALARCON HERRERA ESTEBAN JAVIER ANTONIO

ERIX ALEJANDRO SOLARES FLORES

ERICK ROLANDO SAPON PORTELA

Enlace Drive:

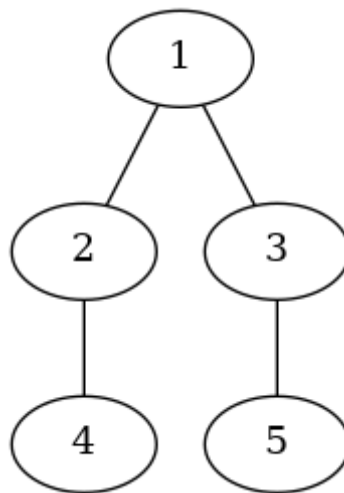
<https://drive.google.com/drive/folders/1jdZ-0RVsfPojxcUdbOBHzAXGneqhOFdJ?usp=sharing>

Enlace GitHub:

<https://github.com/ericksapon1/Tarea-XII-BFS-y-DFS>

Parte 1 – Recorridos manuales de grafos

Ejercicio 1:

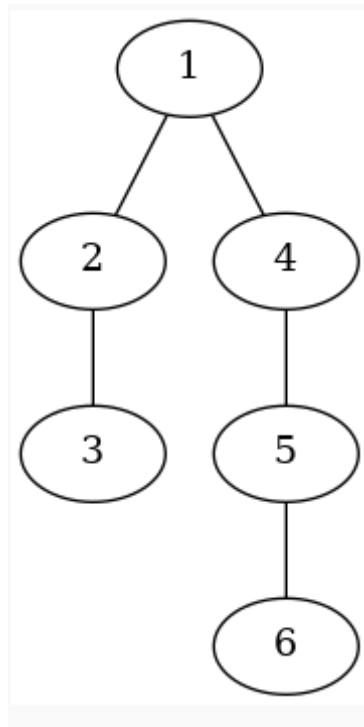


Lista de Adyacencia: 1(2,3) 2(4) 3(5)

BFS: 1,2,3,4,5

DFS: 1,2,4,3,5

Ejercicio 2

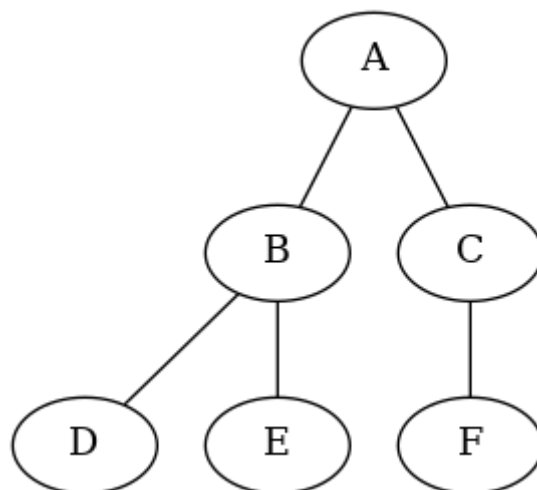


Lista de adyacencia 1(2, 4), 2(3), 4(5).

BFS: 1, 2, 4, 3, 5, 6

DFS: 1, 2, 3, 4, 5, 6

Ejercicio 3:

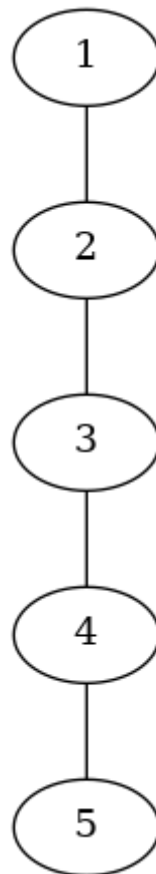


Lista de Adyacencia: A(B, C), B(D, E), C(F).

BFS: A, B, C, D, E, F

DFS: A, B, D, E, C, F

Ejercicio 4:

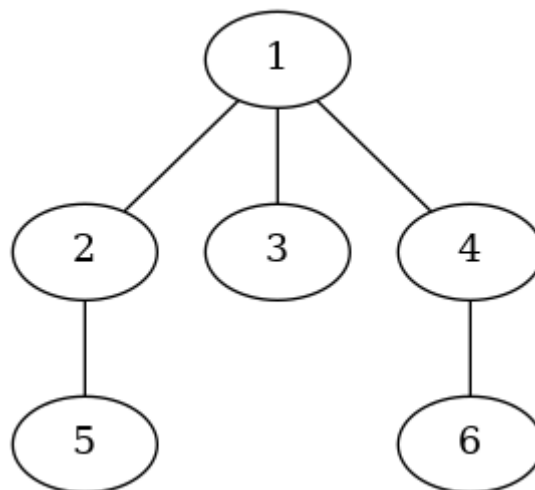


Lista de adyacencia: 1(2) 2(3) 3(4) 4(5)

BFS: 1,2,3,4,5

DFS: 1,2,3,4,5

Ejercicio 5:

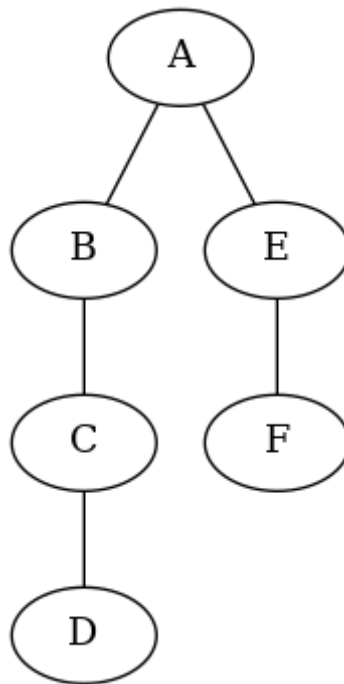


Lista de adyacencia 1(2, 3, 4), 2(5), 4(6).

BFS: 1, 2, 3, 4, 5, 6

DFS: 1, 2, 5, 3, 4, 6

Ejercicio 6:

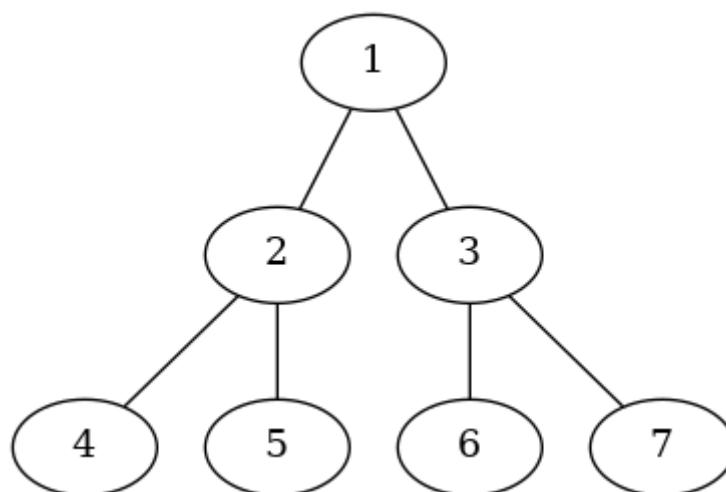


Lista de adyacencia: A(B, E), B(C), C(D), E(F).

BFS: A, B, E, C, F, D

DFS: A, B, C, D, E, F

Ejercicio 7:

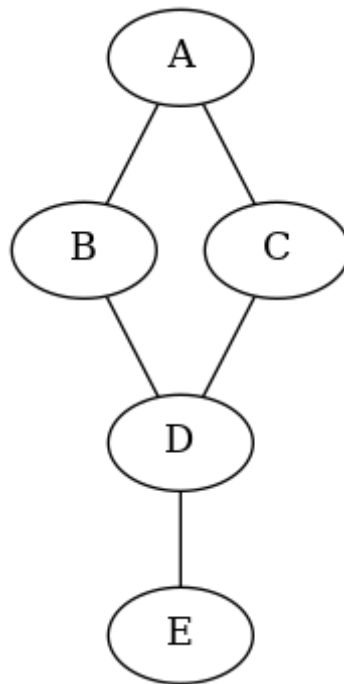


Lista de adyacencia: 1(2,3) 2(4,5) 3(6,7)

BFS: 1,2,3,4,5,6,7

DFS: 1,2,4,5,3,6,7

Ejercicio 8:

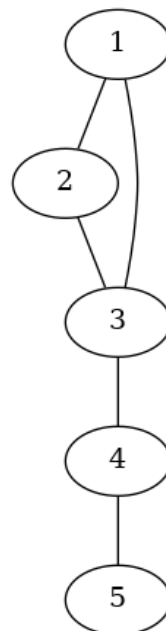


Lista de adyacencia: A(B, C), B(D), C(D), D(E).

BFS: A, B, C, D, E

DFS: A, B, D, E, C

Ejercicio 9:

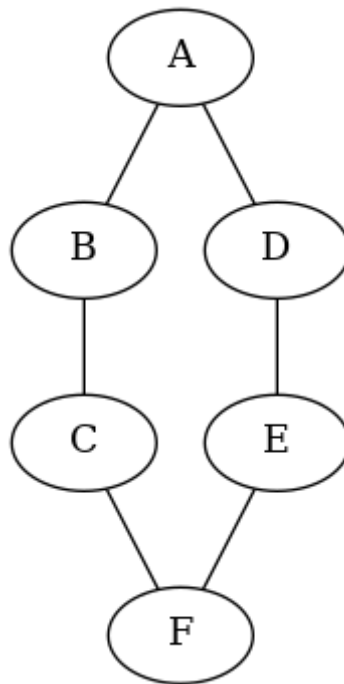


Lista de adyacencia 1(2, 3), 2(3), 3(1, 4), 4(5)

BFS: 1, 2, 3, 4, 5

DFS: 1, 2, 3, 4, 5

Ejercicio 10:

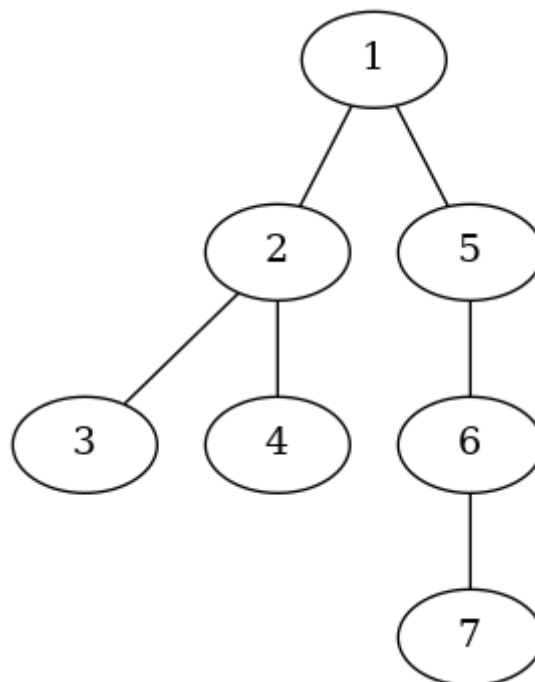


Lista de adyacencia: A(B,D) B(C) D(E) C(F) E(F)

BFS: A,B,D,C,E,F

DFS: A,B,C,F,D,E

Ejercicio 11:

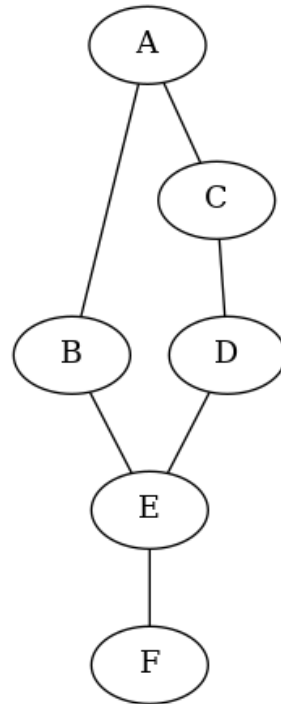


Lista de adyacencia 1(2, 5), 2(3, 4), 5(6), 6(7).

BFS: 1, 2, 5, 3, 4, 6, 7

DFS: 1, 2, 3, 4, 5, 6, 7

Ejercicio 12:

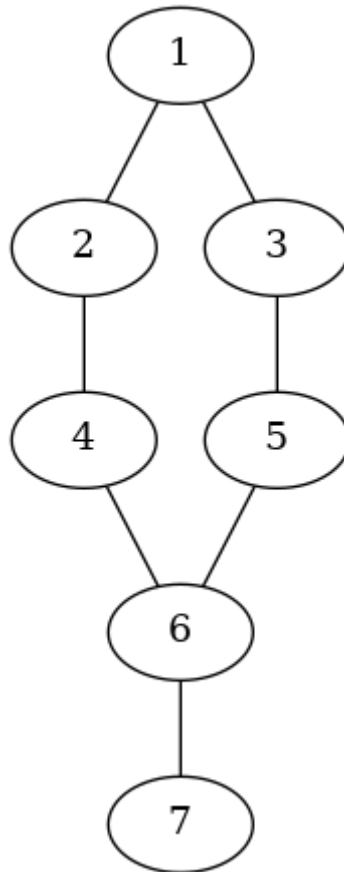


Lista de adyacencia: A(B,C) B(E) C(D) D(E) E(F)

BFS: A,B,C,E,D,F

DFS: A,B,E,F,C,D

Ejercicio 13:

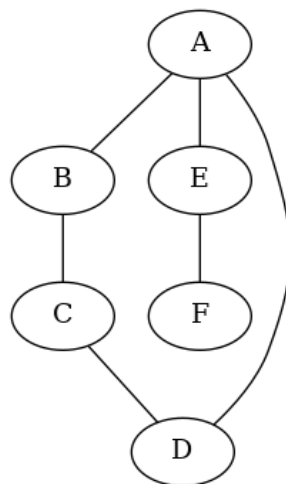


Lista de adyacencia: 1(2,3) 2(4) 3(5) 4(6) 5(6) 6(7)

BFS: 1,2,3,4,5,6,7

DFS: 1,2,4,6,7,3,5

Ejercicio 14:

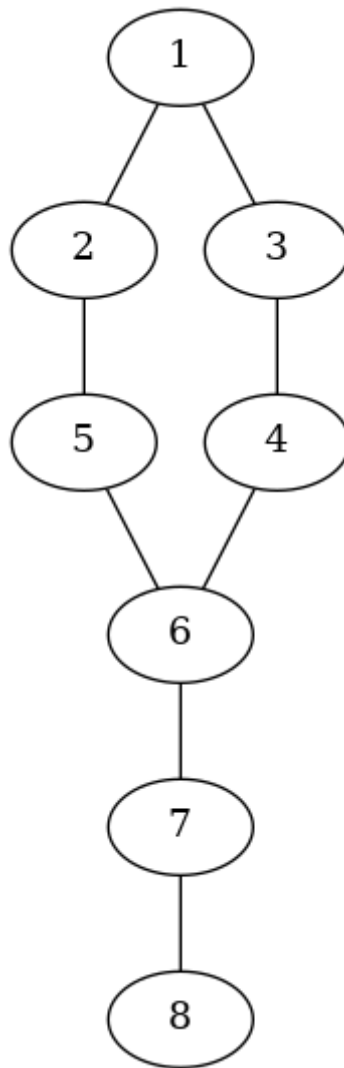


Lista de adyacencia A(B, E), B(C), C(D), D(A), E(F).

BFS: A, B, E, C, F, D

DFS: A, B, C, D, E, F

Ejercicio 15:

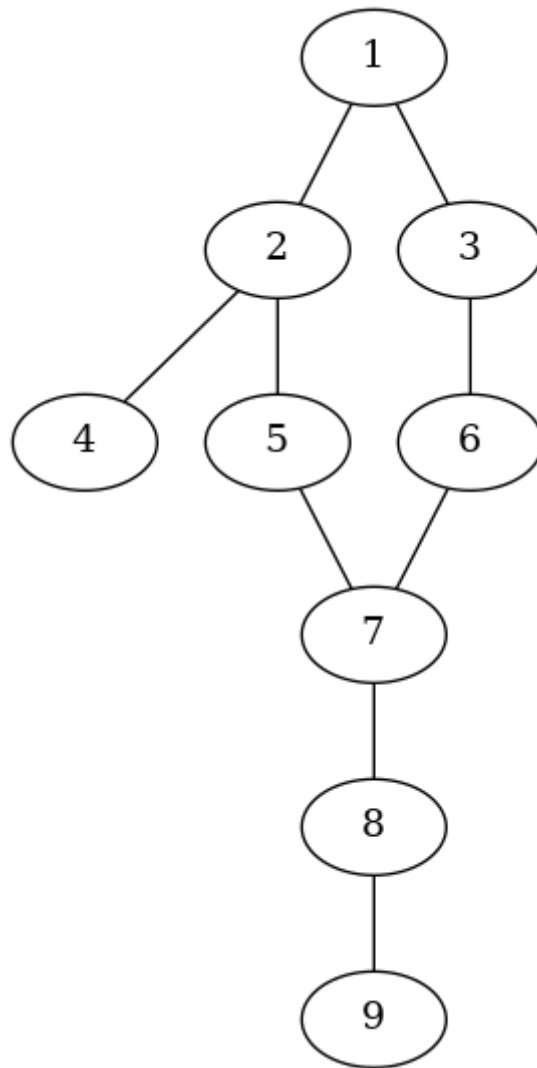


Lista de adyacencia: 1(2,3) 2(5) 3(4) 4(6) 5(6) 6(7) 7(8)

BFS: 1,2,3,5,4,6,7,8

DFS: 1,2,5,3,4,6,7,8

Ejercicio 16



Lista de adyacencia: 1(2,3) 2(4,5) 3(6) 5(7) 6(7) 7(8) 8(9)

BFS: 1,2,3,4,5,6,7,8,9

DFS: 1,2,4,5,7,8,9,3,6

1. ¿En qué momento (nivel) BFS llega al nodo 9?
Al llegar al nivel 6.
2. ¿DFS visita el nodo 9 antes o después que BFS?
Lo visita antes.
3. ¿Cuál recorrido considera más eficiente en este caso y por qué?
En este caso si es para encontrar el nodo 9 el mejor recorrido es DFS

Parte 2 – Comparación técnica C++ vs Java

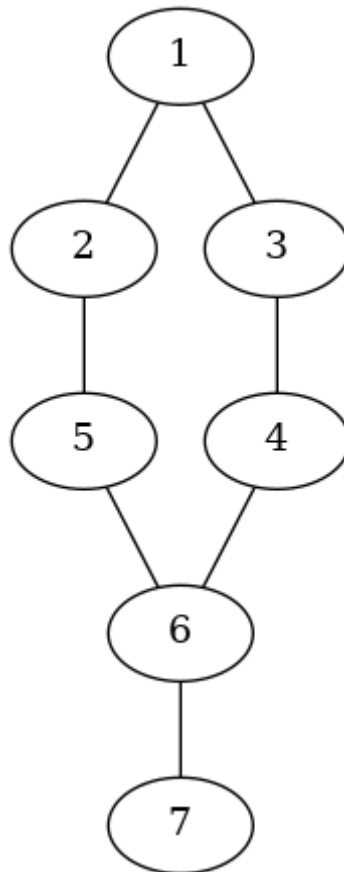
Instrucciones

El grupo deberá ejecutar dos implementaciones funcionales del mismo grafo:

Una versión en C++ puro, sin usar vector, queue, stack, map ni unordered_map.

Una versión en Java, usando HashMap, ArrayList, Queue y LinkedList.

Luego deberán elaborar una comparación técnica.



Análisis Técnico

1. ¿Qué estructuras se implementaron manualmente en C++?

Se construyeron desde cero:

- Listas enlazadas simples: A través de la estructura `NodoAdyacente` para manejar las listas de adyacencia.
- Cola (Queue): Implementada con punteros (`frente` y `final`) y la estructura `NodoCola` para el control FIFO del BFS.
- Arreglos dinámicos y Set manual: El arreglo de `Vertice` y el arreglo booleano de visitados (`bool* visitado = new bool[cantidad]`).

2. ¿Qué estructuras ya existen en Java?

Se utilizó el *Java Collections Framework* (JCF):

- `Map` (`HashMap`) para mapear el ID del vértice con sus vecinos.
- `List` (`ArrayList`) para almacenar la lista de adyacencia de cada vértice.
- `Set` (`HashSet`) para el registro de nodos visitados con búsqueda en $O(1)$.
- `Queue` (`LinkedList`) para la gestión FIFO en BFS.

3. ¿Cuál código es más largo?

C++ es considerablemente más largo. Requiere escribir el *boilerplate* estructural (creación de nodos, enlaces de punteros) y el código de mantenimiento de memoria (destructores para evitar *memory leaks*), duplicando la cantidad de líneas en comparación con Java.

4. ¿Cuál código es más fácil de entender?

Java. Su alto nivel de abstracción permite una programación más declarativa. La lectura del algoritmo BFS o DFS se enfoca puramente en la lógica del recorrido matemático de grafos, ocultando la "fontanería" de los punteros y la gestión del *Heap*.

5. ¿Cuál lenguaje exige mayor control del programador?

C++. Obliga al programador a gestionar manualmente el ciclo de vida de la memoria (con `new` y `delete`). Un puntero mal referenciado causará un *Segmentation Fault*, y un objeto no destruido causará una fuga de memoria.

6. ¿Qué ventajas tiene C++ en este caso?

- Rendimiento puro y huella de memoria: Al no existir el *Garbage Collector* en segundo plano ni el *overhead* de los objetos de Java, la ejecución es estrictamente determinista y consume un mínimo de RAM.
- Ausencia de "Boxing/Unboxing": C++ opera directamente con el tipo primitivo `int`. En Java, el uso de genéricos (`List<Integer>`) obliga al runtime a envolver y desenvolver constantes primitivas en objetos `Integer`, lo cual penaliza el rendimiento y dispersa la memoria caché.

7. ¿Qué ventajas tiene Java en este caso?

- Velocidad de desarrollo y seguridad: El *Garbage Collector* elimina el riesgo de fugas de memoria.
- Robustez estructural: Las colecciones integradas están exhaustivamente probadas contra *edge cases* y optimizadas a nivel de la Máquina Virtual de Java (JVM), reduciendo la superficie de errores lógicos del programador.

8. ¿Qué sucede si en C++ no se libera memoria correctamente?

Se produce un Memory Leak (fuga de memoria). La memoria asignada en el *Heap* queda inaccesible pero marcada como ocupada por el sistema operativo. En un proceso de larga ejecución (como un demonio o un servidor), el consumo de RAM escalará hasta agotar la capacidad física, provocando que el sistema operativo intervenga (vía *OOM Killer*) destruyendo el proceso, o provocando un cuelgue general del sistema.

9. ¿Por qué BFS necesita una cola?

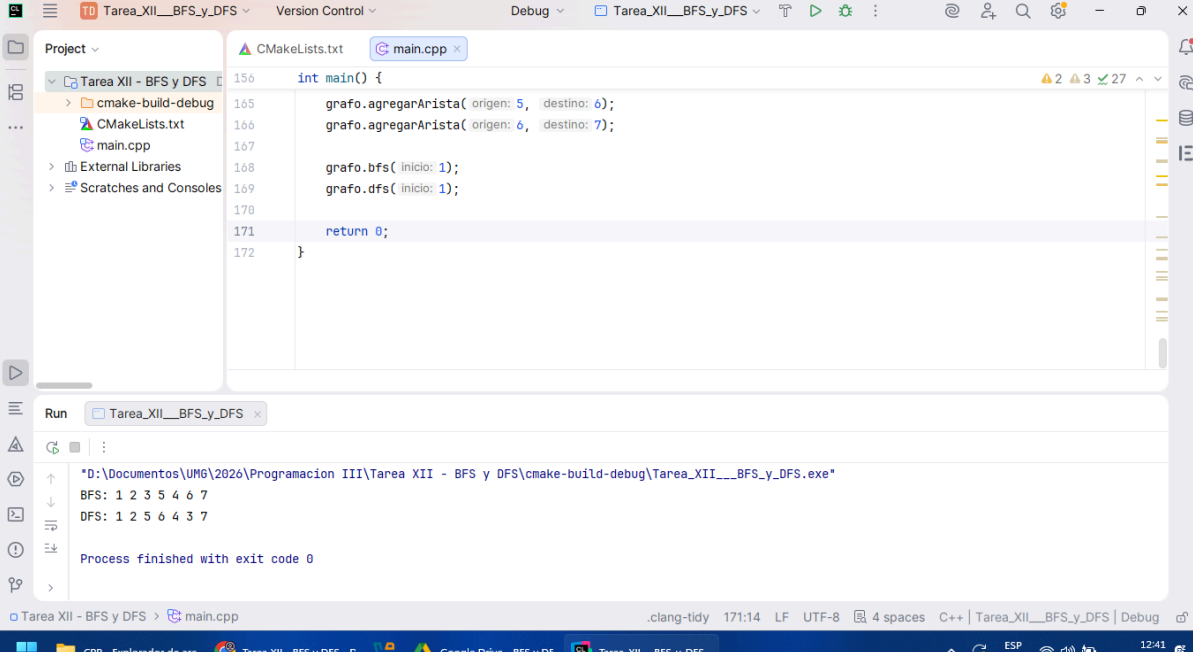
Porque BFS explora el grafo por niveles (anchura). La topología requiere un comportamiento FIFO (First-In, First-Out). Una cola garantiza que los primeros nodos descubiertos (más cercanos a la raíz) se procesen y expandan antes de pasar a los nodos descubiertos en el siguiente nivel de profundidad.

10. ¿Por qué DFS puede resolverse con recursión?

Porque DFS requiere un comportamiento LIFO (Last-In, First-Out) para explorar profundizando por una rama hasta llegar a un nodo hoja antes de hacer *backtracking*. La recursividad utiliza el Call Stack (pila de llamadas del sistema) como su estructura LIFO nativa. Cada llamada recursiva apila el estado del nodo actual, permitiendo retomar la exploración de los vecinos restantes cuando se desapila (retorna).

Evidencias (Capturas de ejecución):

C++



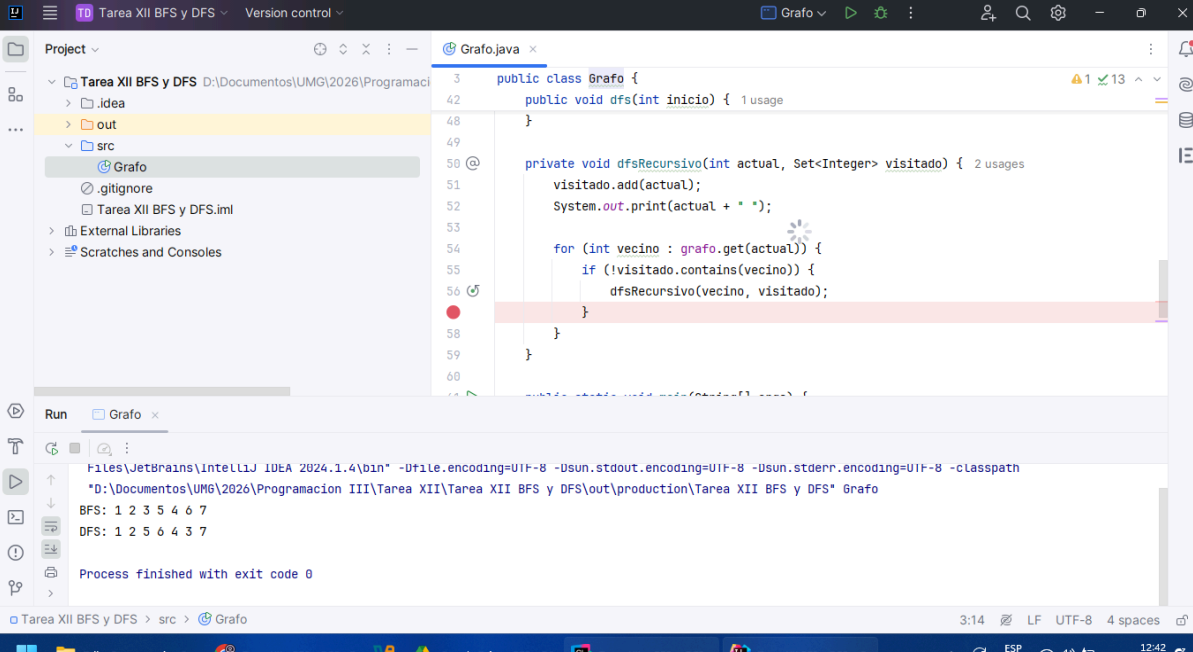
The screenshot shows the Visual Studio Code interface with a C++ project named "Tarea_XII__BFS_y_DFS". The main.cpp file is open, showing the following code:

```
156 int main() {
165     grafo.agregarArista( origen: 5, destino: 6);
166     grafo.agregarArista( origen: 6, destino: 7);
167
168     grafo.bfs( inicio: 1);
169     grafo.dfs( inicio: 1);
170
171     return 0;
172 }
```

The Run window shows the execution output:

```
*D:\Documentos\UMG\2026\Programacion III\Tarea XII - BFS y DFS\cmake-build-debug\Tarea_XII__BFS_y_DFS.exe*
BFS: 1 2 3 5 4 6 7
DFS: 1 2 5 6 4 3 7
Process finished with exit code 0
```

Java



The screenshot shows the IntelliJ IDEA interface with a Java project named "Tarea XII BFS y DFS". The Grafo.java file is open, showing the following code:

```
3 public class Grafo {
42     public void dfs(int inicio) { 1 usage
48     }
49
50     private void dfsRecursivo(int actual, Set<Integer> visitado) { 2 usages
51         visitado.add(actual);
52         System.out.print(actual + " ");
53
54         for (int vecino : grafo.get(actual)) {
55             if (!visitado.contains(vecino)) {
56                 dfsRecursivo(vecino, visitado);
57             }
58         }
59     }
60 }
```

The Run window shows the execution output:

```
Files\JetBrains\IntelliJ IDEA 2024.1.4\bin\ -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath
"D:\Documentos\UMG\2026\Programacion III\Tarea XII\Tarea XII BFS y DFS\out\production\Tarea XII BFS y DFS" Grafo
BFS: 1 2 3 5 4 6 7
DFS: 1 2 5 6 4 3 7
Process finished with exit code 0
```